

Bibliography

- [1] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus, “Fine-grained and accurate source code differencing.”
- [2] Dyer, Robert, Rajan, Hridesh, Nguyen, Hoan Anh, and Nguyen, Tien N., “Mining billions of AST nodes to study actual and potential usage of Java language features,” *Association for Computing Machinery*.
- [3] Lämmel, Ralf, Pek, Ekaterina, and Starek, Jürgen, “Large-scale, AST-based API-usage analysis of open-source Java projects.”
- [4] Arafat, O. and Riehle, D., “The Commit Size Distribution of Open Source Software.”

Things to know when building tools that work on code

Why is this not common knowledge?

MARKO IVANKOVIĆ¹

¹marko@ivankovic.me, Universität Passau

BMW

München, 2026-03-04

Overview

1. Background
2. Existing research
3. Let's dig
4. Rules of thumb
5. What am I doing next?

BACKGROUND

Background

- As an industry, we make a lot of tools for Software Engineers.
- Many of these tools work on code.
- But do we know how code **actually** looks like?

Why does this matter?

- How often did you see the “worst case” “upper bound” Big-O notation in papers?
- As an example, in “Fine-grained and Accurate Source Code Differencing” [1], the authors state that the complexity of their algorithm is $O(n^2)$ where n is the maximum of the number of nodes in the two trees being compared.
- What **is** the maximum number of nodes?
 - What is the theoretical maximum number of nodes?
 - What is the physically possible maximum number of nodes?
 - What is the actual maximum number of nodes on the Planet Earth today?
 - What is the maximal number of nodes your tool will ever see?
 - What is the second largest number? Or how about 99.999th percentile?

Difference between engineering and science

- If civil engineers built houses the way we build software, every house would be designed to survive a volcanic eruption happening at the same time as a nuclear blast... underwater because a tsunami just passed over the area.
- When you build a house, the first step is to get a geological engineer to do ground analysis and then you design your house based on actual constraints, not based on possible constraints.
- Very often, we build the tool first, using theoretical “worst case” analysis to guide the development, and then we measure the actual performance **after** the tool is built.

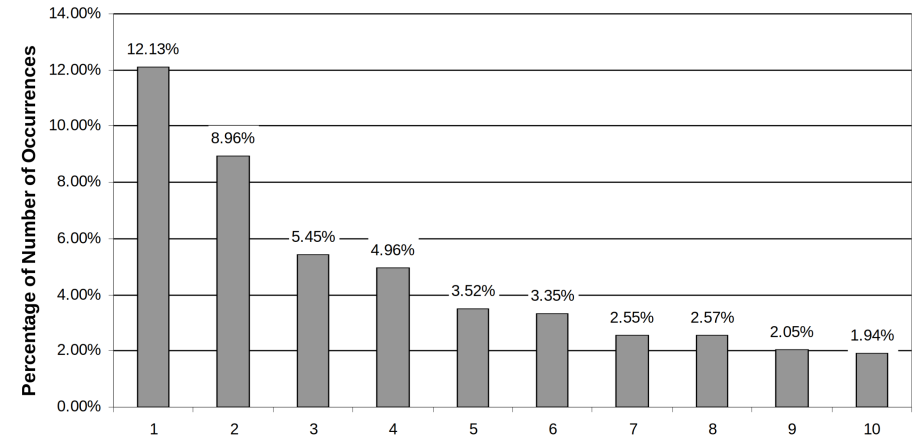
EXISTING RESEARCH

Two separate papers that use AST nodes to look for API usage

- “Mining Billions of AST Nodes to Study Actual and Potential Usage of Java Language Features” [2] and “Large-scale, AST-based API-usage analysis of open-source Java projects” [3] both use AST analysis to find function calls and analyze API usage
- Neither provides a comprehensive analysis of size, except for offhand sentences like “In this paper, we analyze over 31k opensource Java projects representing over 9 million Java files, which when parsed contain over 18 billion AST nodes”
- About 2 thousand nodes per file on average then :)
- Java only

The Commit Size Distribution of Open Source Software [4].

- Best paper I found so far for commit sizes
- 8M commits across 9363 projects
- **Key-takeaway:** Distribution of commit sizes is logarithmic.
 - 47.5% of all commits are 10 or fewer lines.
 - Commits beyond 10000 lines do exist.
- **Some limitations:**
 - Doesn't provide a per-language overview
 - LOC focus, not AST nodes



LET'S DIG

Let's find the answer to our questions

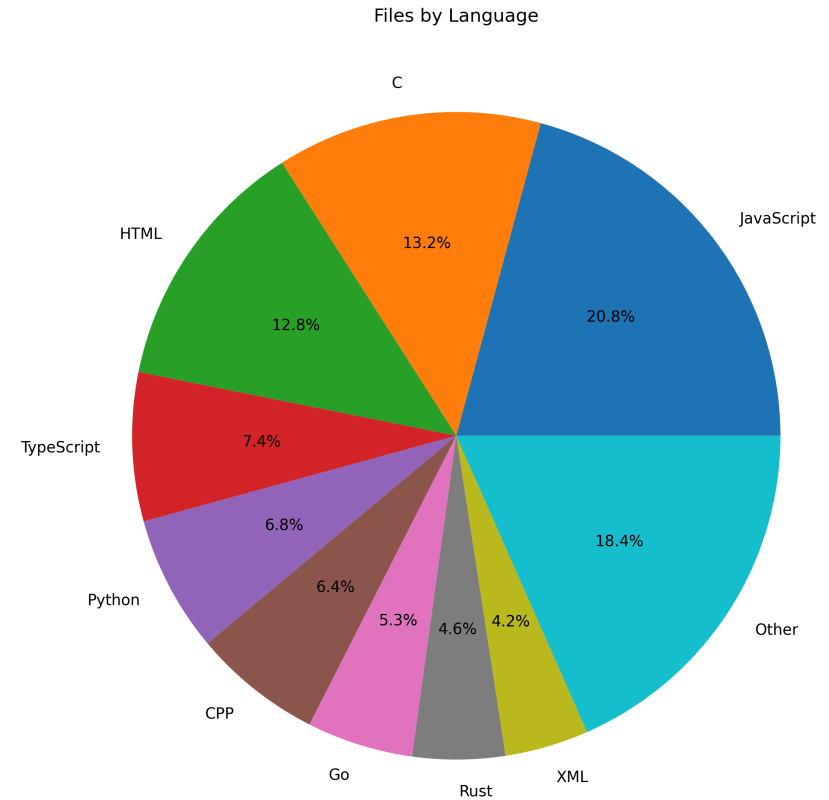
- What is the theoretical maximum number of nodes?
 - Ok this is probably ∞ .
- What is the physically possible maximum number of nodes?
 - There are 10^{80} atoms in the universe. So if we converted the entire universe into RAM... A lot, but infinitely fewer than ∞ .
- What is the actual maximum number of nodes on the Planet Earth today?
- What is the maximal number of nodes your tool will ever see?
- What is the second largest number? Or how about 99.999th percentile?

What is the actual maximum number of nodes on the Planet Earth today?

- We can't know this. Too many private repositories. But we can make a decent educated guess.
- Let's take the top 1000 open source repositories, let's add the roughly 7300 more repositories that make up the Gentoo Linux distribution.
- Let's clone them all and count.

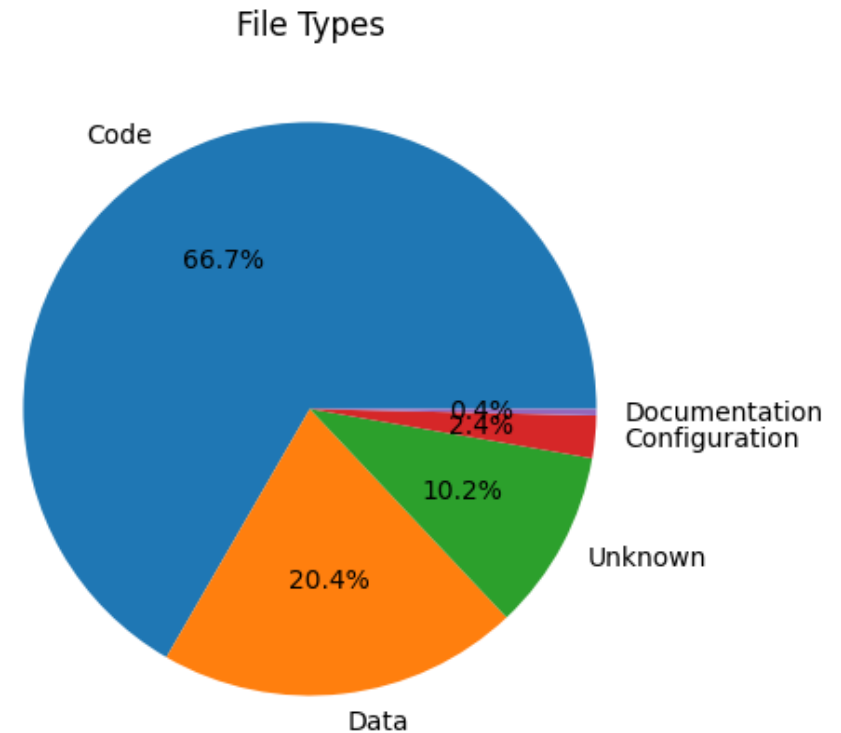
Attack of the clones

- Repositories: 100
- Number of files: 1347492
- Languages: 31
- 4 **types** of files



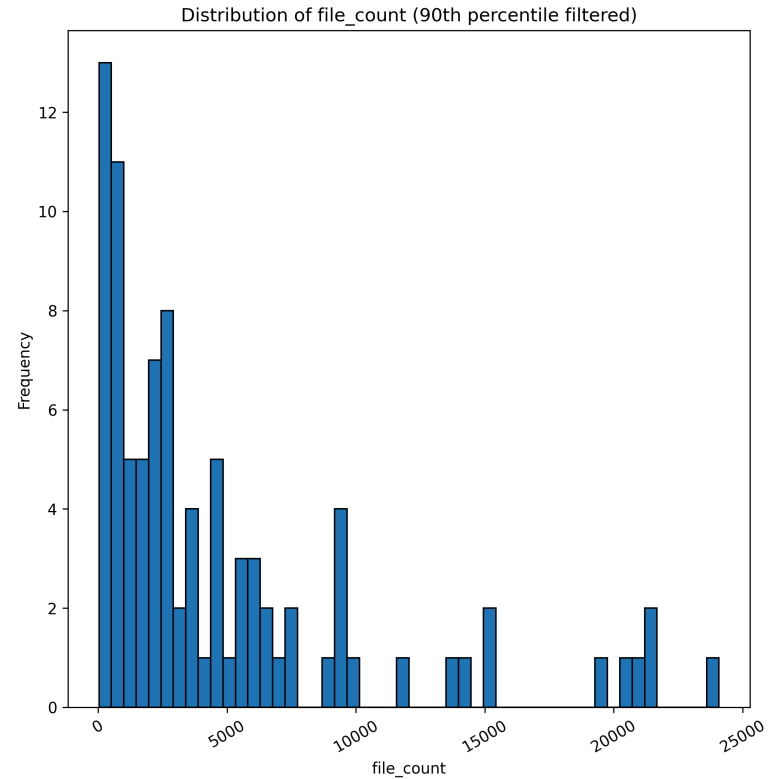
Repositories contain a lot of stuff, not just code

- Code
 - This is what you expect to find
- Configuration
 - Usually human readable. Yaml and similar formats
 - A lot of it is configuring the development environment
- Data
 - Images, audio, video...
 - Test data
 - Data stored as very long arrays directly in code
- Documentation
 - Project guides, e.g. README.md
 - Legal, e.g. licences



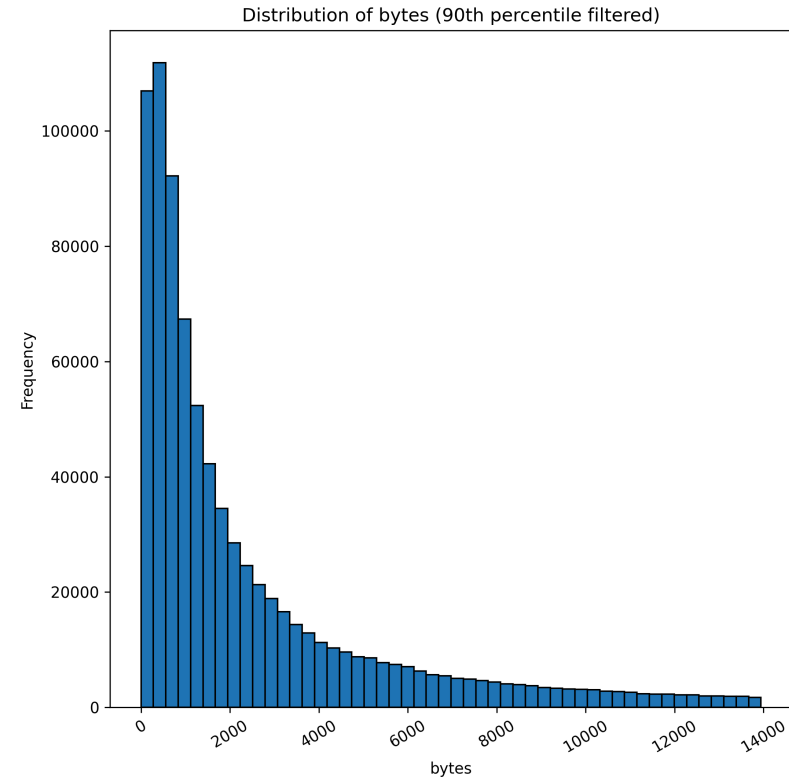
Files per project

- Median:
- 90th percentile:
- 99th percentile:
- 99.9th percentile:



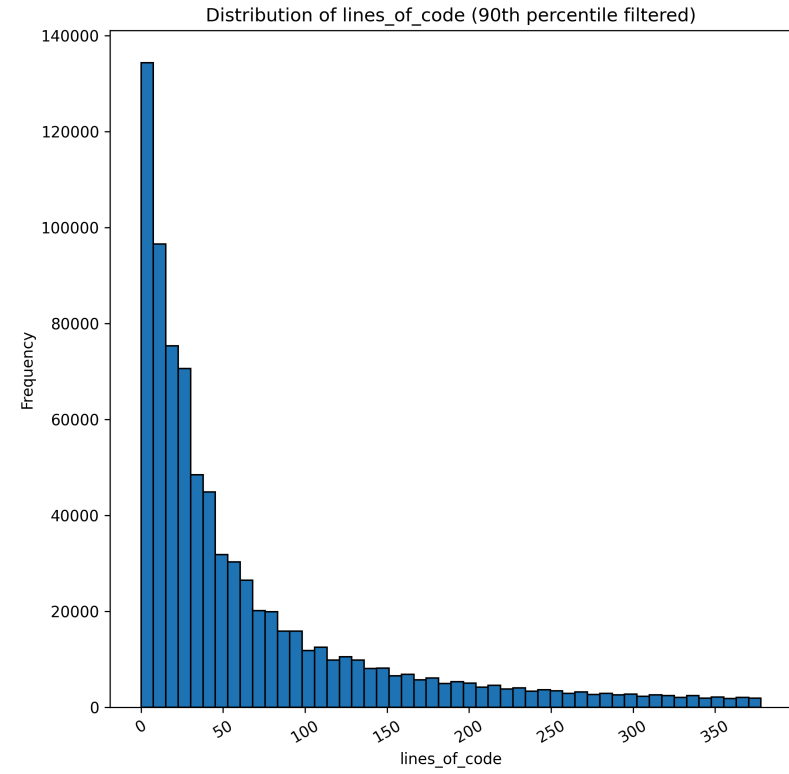
Bytes per file (Code files only)

- Median: 1508
- 90th percentile:
- 99th percentile:
- 99.9th percentile:



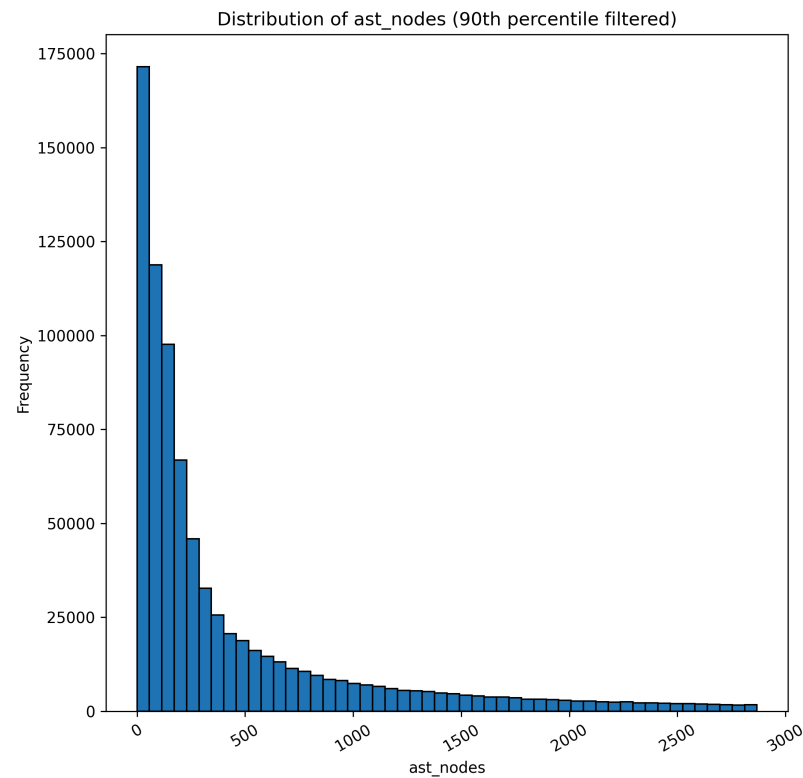
LOC per file (Code files only)

- Median: 1508
- 90th percentile:
- 99th percentile:
- 99.9th percentile:



AST nodes per file (Code files only)

- Median: 1508
- 90th percentile:
- 99th percentile:
- 99.9th percentile:



Correlation between bytes and AST nodes (Code files only)

- Pearson correlation: 0.8793

- Formula:

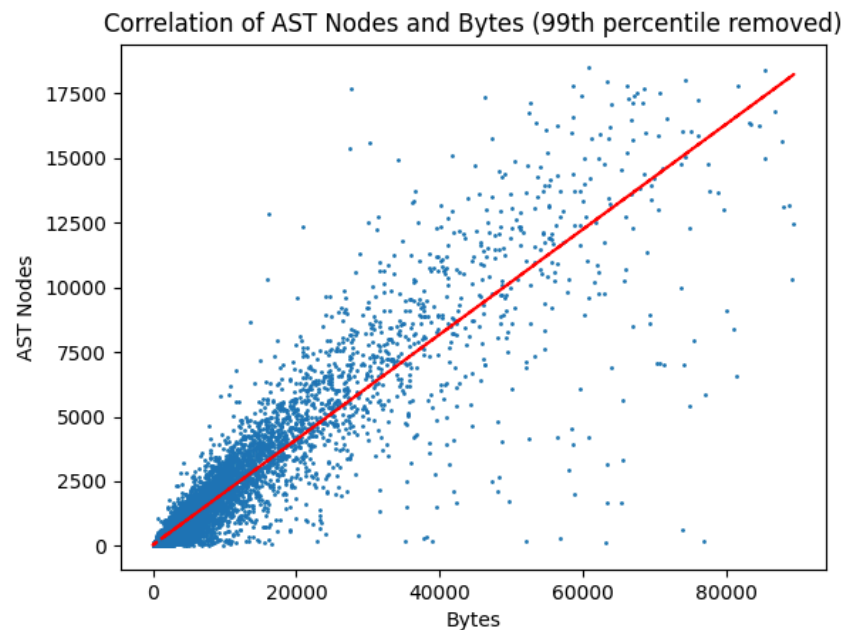
$$|AST| = \text{bytes} * 0.21128 + 18.78731$$

- More useful formula:

$$|AST| = \frac{\text{bytes}}{4}$$

- This allows comparing algorithms that operate on bytes and AST nodes!

‣ E.g. $O(\text{bytes}^2) = O(|AST|^2)$



RULES OF THUMB

Quick rules of thumb when time is precious

- **Repository composition**: Two-thirds code, one third other stuff.
- **Files per project**: Tens of thousands mostly, can be hundreds of thousands.
- **Bytes per file**: Tens of thousands mostly, can be half a million. Million is rare.
- **AST nodes per file**: Number of bytes divided by 4. Thousands to tens of thousands. Quarter million is rare.
- **Commit size**: Half of all commits is under 10 lines. But they go up to tens of thousands of lines.

WHAT AM I DOING NEXT?

CodeDiff

- I really hate how code diffing works today. If you have ever seen a GitHub diff where somebody inlines Python code in a new function, you know what annoys me.
- But there are better theorethical diffing algorithms.
- But none of them are implemented with usage-first mentality. Most are research-first and explicitly say that they are not aiming to be fast and robust.
- I want a fast, robust, AST based diff!
- Theorethical algorithms are $O(|AST|^2)$. But we now know, the actual sizes are such that it would be much better to design something that works with tens of thousands of nodes but only 10-100 changed lines!

It is easier to make a **robust** tool if you know the real constraints.

Thank you for your attention

marko@ivankovic.me